Use Case 6: Financial Transaction Fraud Detection (Lambda Architecture)

1. Overview

Build a **Lambda Architecture** for financial transaction fraud detection using **Snowflake**, **PySpark**, **AWS Glue**, and **Apache Kafka**. The system combines historical batch analysis with real-time streaming to detect fraudulent transactions as they happen, while maintaining a comprehensive view of all transaction data.

The Lambda Architecture has three layers:

- **Batch Layer**: Processes historical data in bulk to build account risk profiles
- **Speed Layer**: Processes live transactions in real time, comparing them against risk profiles to flag suspicious activity
- **Serving Layer**: Combines outputs from both layers into unified views for analysts and downstream systems

---

2. Business Scenario

SecureBank processes millions of financial transactions daily. Their fraud detection team needs:

> **Historical analysis**: Load and analyze past transactions to understand each account's "normal" behavior -- average spending, typical locations, preferred merchants, and transaction frequency.
>
> **Real-time detection**: As new transactions stream in, compare them against established account profiles. Flag transactions that deviate significantly from the norm: unusually large amounts, transactions from unexpected locations, or rapid-fire purchases that suggest a stolen card.
>
> **Unified reporting**: Analysts need a single view that combines historical patterns with real-time alerts, allowing them to prioritize investigations and track fraud trends.

The solution must handle the inherent tension in fraud detection: **batch processing is thorough but slow; stream processing is fast but has limited context**. The Lambda Architecture resolves this by running both simultaneously and merging their outputs.

---

3. Learning Objectives

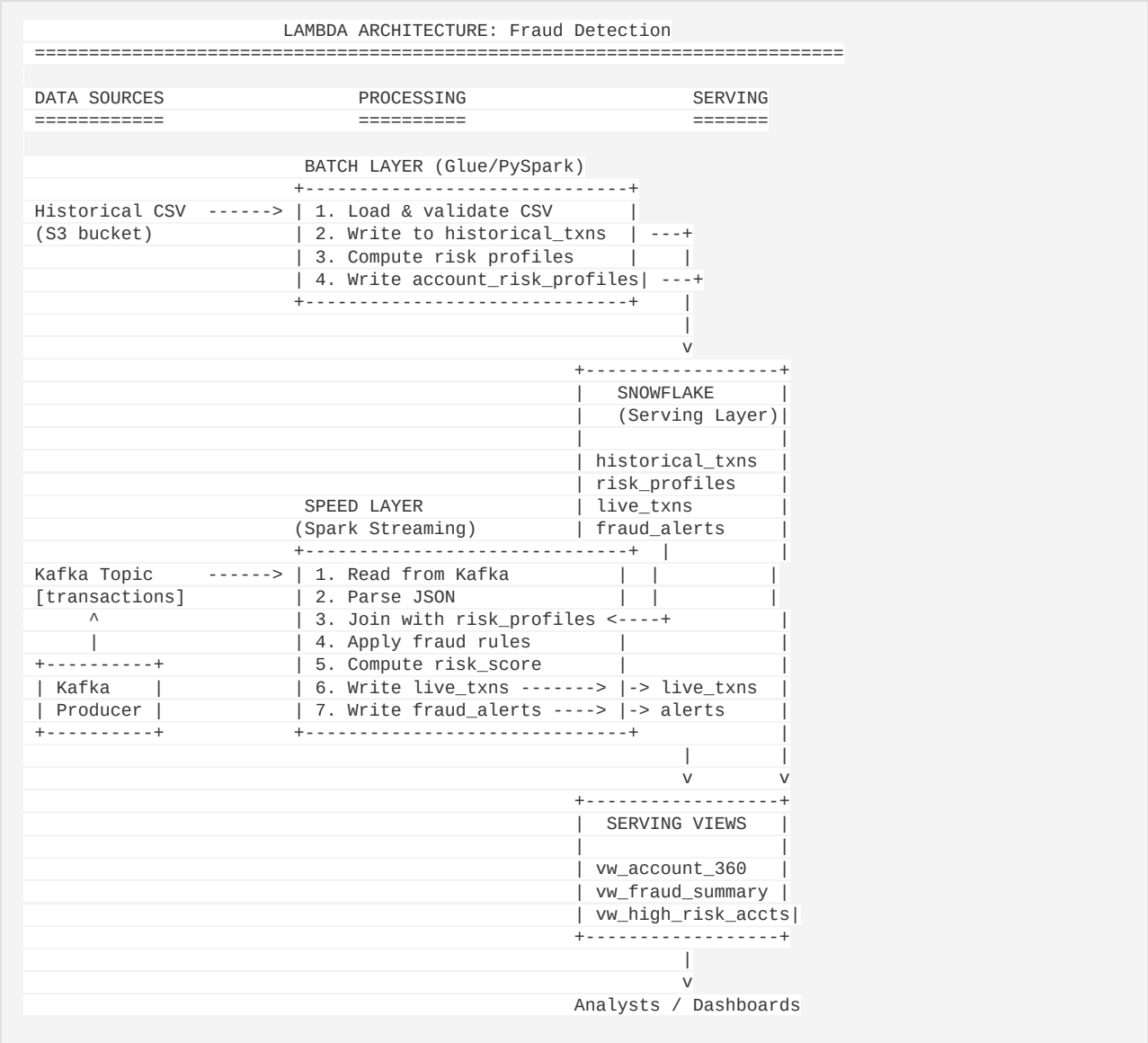
By completing this use case, you will be able to:

- Explain the **Lambda Architecture** pattern and when to use it
- Distinguish between **batch layer** and **speed layer** responsibilities
- Build a **batch pipeline** with AWS Glue/PySpark that loads data and computes aggregations
- Build a **Spark Structured Streaming** pipeline that reads from Kafka
- Implement **rule-based fraud detection** using window functions and conditional logic
- Perform **broadcast joins** to enrich streaming data with reference data
- Use Snowflake **VARIANT** columns to store semi-structured data

- Create **serving layer views** that unify batch and speed layer results
- Write **MERGE** (upsert) operations in Snowflake
- Design **risk scoring** systems with weighted, multi-rule evaluation

---

4. Architecture Diagram



---

5. Dataset Description

5.1 Historical Transactions CSV

File: `datasets/use_case_6/historical_transactions.csv` **Records**: ~200 rows covering January-June 2024

Column	Type	Description
transaction_id	String	Unique ID (TXN_00001 to TXN_00200)

account_id	String	Account (ACC_001 to ACC_050), multiple txns each
amount	Decimal	Transaction amount ($1.50 to $5,000)
merchant	String	Merchant name (Amazon, Walmart, Shell Gas, etc.)
category	String	One of: Grocery, Gas, Restaurant, Online Shopping, Travel, Entertainment, ATM Withdrawal
city	String	Transaction city
state	String	Transaction state
country	String	Transaction country (mostly US)
timestamp	Timestamp	When the transaction occurred
is_fraud	Integer	Ground truth label: 0 = legitimate, 1 = fraud

Key characteristics: - ~5% fraud rate (approximately 10 fraudulent transactions) - Fraudulent transactions tend to: have large amounts ($2,000+), originate from unusual international locations, occur during late-night hours (2-4 AM) - Each account has a "home" city where most of its transactions occur - Normal transactions cluster in the $15-$200 range

5.2 Live Transactions JSON (Kafka Messages)

File: `datasets/use_case_6/sample_live_transactions.json` **Format**: JSON Lines (one JSON object per line) **Records**: 20 sample transactions

Same schema as the CSV **except no `is_fraud` column** -- this is what the system must detect. Notable patterns in the sample data: - TXN_L0001: Large amount ($2,499.99) from ACC_003 in Miami (normally in San Francisco) - TXN_L0004: New account (ACC_051), large foreign transaction - TXN_L0007: ACC_004 with $3,200 from Romania (normally in Houston) - TXN_L0011 through TXN_L0016: Rapid-fire burst from ACC_003 (6 transactions in ~5 minutes) - TXN_L0019: New account (ACC_053), large foreign transaction from India

---

6. Tasks

Task 1: Set Up Snowflake Tables

Create the complete Snowflake schema for the Lambda Architecture.

Batch Layer Tables: - `historical_transactions`: Stores labeled historical data - `account_risk_profiles`: Pre-computed per-account behavioral baselines

Speed Layer Tables: - `live_transactions`: Incoming real-time transactions with risk scores - `fraud_alerts`: Flagged transactions with rule details

Serving Layer Views: - `vw_account_360`: Combined account view (batch profile + live activity + alerts) - `vw_fraud_summary`: Hourly fraud alert aggregations - `vw_high_risk_accounts`: Accounts ranked by combined risk signals

Deliverable: SQL script that creates all objects. Pay attention to: - Using `VARIANT` type for semi-structured data (locations array, categories array, alert details) - Appropriate primary keys and clustering - A stored procedure for refreshing risk profiles

---

Task 2: BATCH -- Load Historical Transactions into Snowflake via Glue

Write an AWS Glue batch job (PySpark) that:

1. Reads `historical_transactions.csv` from S3
2. Validates the data:
3. Drop rows with null transaction_id, account_id, or amount
4. Parse timestamps (handle both `yyyy-MM-dd HH:mm:ss` and ISO 8601 formats)
5. Ensure amounts are positive
6. Default `is_fraud` to 0 where null
7. Trim string fields
8. Deduplicate by transaction_id
9. Produces a data quality report (counts, date range, fraud rate, category distribution)
10. Writes cleaned data to Snowflake `historical_transactions` table

Key Spark concepts: schema definition, data validation, `withColumn`, `filter`, `dropDuplicates`

Task 3: BATCH -- Compute Account Risk Profiles

Extend the batch job to compute per-account risk profiles:

Metric	Computation
total_txns	COUNT(*)
avg_amount	AVG(amount)
max_amount	MAX(amount)
std_dev_amount	STDDEV(amount)
common_locations	collect_list of DISTINCT (city, state, country)
common_categories	collect_list of DISTINCT category
total_fraud_count	SUM(is_fraud)

fraud_rate	total_fraud_count / total_txns
first_txn_date	MIN(timestamp)
last_txn_date	MAX(timestamp)

Write risk profiles to Snowflake `account_risk_profiles` table.

Key Spark concepts: `groupBy`, `agg`, `collect_list`, `struct`, `to_json`, joins

Task 4: Start Kafka and Run Transaction Producer

1. Start your Kafka cluster (or use a managed service)
2. Create the `transactions` topic: `bash kafka-topics.sh --create --topic transactions --bootstrap-server localhost:9092 \ --partitions 3 --replication-factor 1`
3. Run the transaction producer: `bash python streaming_utils/kafka_producer_transactions.py \ --bootstrap-servers localhost:9092 \ --topic transactions \ --num-accounts 50 \ --tps 3 \ --fraud-rate 0.05`
4. Verify messages are flowing: `bash kafka-console-consumer.sh --topic transactions --bootstrap-server localhost:9092 \ --from-beginning --max-messages 5`

Task 5: STREAM -- Read Live Transactions from Kafka

Write a Spark Structured Streaming application that:

1. Connects to Kafka and subscribes to the `transactions` topic
2. Reads messages with `startingOffsets = "latest"`
3. Parses the JSON value into a structured DataFrame using `from_json`
4. Converts the timestamp string to a proper `TimestampType`

Key Spark concepts: `readStream`, `format("kafka")`, `from_json`, schema definition

Task 6: STREAM -- Enrich with Account Risk Profiles (Broadcast Join)

In your `foreachBatch` function:

1. Read `account_risk_profiles` from Snowflake (it is a small table)
2. Cache it to avoid repeated reads
3. Use a **broadcast join** to enrich live transactions: `python enriched_df = live_df.join( F.broadcast(risk_profiles), live_df["account_id"] == risk_profiles["profile_account_id"], how="left" )`
4. Handle new/unknown accounts gracefully (left join produces nulls for unknown accounts)

Why broadcast? The risk profiles table is small (~50-200 rows) while the streaming data is unbounded. Broadcasting avoids an expensive shuffle join.

Task 7: STREAM -- Implement Fraud Detection Rules

Apply four fraud detection rules to each enriched transaction:

Rule 1: Amount Threshold - Trigger: `amount > 3 * profile_avg_amount` - Rationale: A transaction 3x the account's historical average is unusual - Score contribution: 25 points

Rule 2: Location Anomaly - Trigger: Transaction city NOT in the account's `common_locations` - Rationale: Transactions from new/unexpected locations may indicate a stolen card - Score contribution: 30 points

Rule 3: Velocity Check - Trigger: More than 5 transactions from the same account within 5 minutes - Rationale: Rapid-fire purchases often indicate automated fraud - Score contribution: 25 points - Implementation: Use a window function with `rangeBetween(-300, 0)` on the timestamp

Rule 4: High Amount from Low-Average Account - Trigger: `amount > $2,000 AND profile_avg_amount < $100` - Rationale: A large purchase from a typically low-spending account is highly suspicious - Score contribution: 20 points

Composite Risk Score: Sum of triggered rule scores (0-100). Transactions with `risk_score > 50` are flagged as fraud alerts.

Task 8: STREAM -- Write Flagged Transactions to fraud_alerts Table

For transactions with `risk_score > 50`:

1. Generate a unique `alert_id`
2. Build the `rule_triggered` string (comma-separated list of rule names)
3. Build the `details` VARIANT column as a JSON object containing:
4. Account profile values (avg, max, std_dev)
5. Transaction count in 5-min window
6. Individual rule flags (0/1)
7. Write to Snowflake `fraud_alerts` table (append mode)

Task 9: STREAM -- MERGE Live Transactions into Snowflake (Upsert)

Write all live transactions (whether flagged or not) to the `live_transactions` table. Use **append mode** for simplicity in streaming, or implement a MERGE pattern:

```
MERGE INTO live_transactions AS tgt
USING (SELECT * FROM incoming_batch) AS src
ON tgt.transaction_id = src.transaction_id
WHEN MATCHED THEN UPDATE SET ...
WHEN NOT MATCHED THEN INSERT ...;
```

Include the `risk_score` and `ingested_at` timestamp in each record.

Task 10: Create Serving Layer Views

Create three views that combine batch and speed layer outputs:

1. **vw_account_360**: For each account, show:

2. Historical profile (avg amount, max amount, known locations)

3. Live transaction summary (count, total amount, max risk score)

Alert summary (total alerts, rules triggered)

vw_fraud_summary: Hourly aggregation of fraud alerts:

6. Alert count per hour

7. Total flagged amount

Breakdown by rule type

vw_high_risk_accounts: Prioritized investigation queue:

10. Risk level classification (CRITICAL / HIGH / MEDIUM / LOW)

11. Combined batch + speed layer signals

12. Sorted by severity

---

7. Expected Outputs

After completing all tasks, you should have:

Component	Location	Description
Snowflake tables (4)	FRAUD_DETECTION_DB.LAMBDA	historical_transactions, account_risk_profiles, live_transactions, fraud_alerts
Snowflake views (3)	FRAUD_DETECTION_DB.LAMBDA	vw_account_360, vw_fraud_summary, vw_high_risk_accounts
Batch Glue job	AWS Glue Console / S3	batch_glue_job.py
Streaming job	Spark cluster	streaming_job.py
Kafka producer	Local / EC2	kafka_producer_transactions.py

Snowflake table row counts (approximate): - `historical_transactions`: ~200 rows - `account_risk_profiles`: ~50 rows (one per account) - `live_transactions`: Growing (depends on how long the stream runs) - `fraud_alerts`: ~5-10% of live transactions

Sample fraud alert (from the sample live data):

```
alert_id     : a1b2c3d4-...
transaction_id: TXN_L0007
account_id   : ACC_004
amount       : 3200.00
rule_triggered: amount_threshold,location_anomaly,high_amount_low_avg
risk_score   : 75
details      : {"profile_avg_amount": 78.45, "profile_max_amount": 250.00, ...}
```

---

8. Hints

Level 1 Hints (Gentle Nudges)

Batch job structure: Think of it as two separate Glue jobs in one script -- first load, then aggregate. The `groupBy("account_id")` is the core of risk profiling.

Streaming enrichment: The key insight is that `account_risk_profiles` is small enough to broadcast. Read it once, cache it, and use it across micro-batches.

Fraud rules: Start with the simplest rule (amount threshold) and add complexity one rule at a time. Use `withColumn` to add rule flags, then sum them for the score.

Serving views: These are just SQL views over the tables your batch and streaming jobs populate. They do not require any Spark code.

Level 2 Hints (More Specific)

Parsing locations for Rule 2: The `common_locations` column is a JSON string. For a quick check, use `F.col("profile_locations").contains(F.col("city"))`. For production quality, parse the JSON array with `from_json` and use `array_contains`.

Velocity check (Rule 3): Use a Spark window function: `python window_spec = Window.partitionBy("account_id") \ .orderBy(F.col("transaction_ts").cast("long")) \ .rangeBetween(-300, 0) df = df.withColumn("txn_count_5min", F.count("*").over(window_spec))`

Risk profile caching: Use a global variable with a TTL to avoid reading from Snowflake on every micro-batch: `python if cache is None or time.time() - cache_time > 300: cache = sf.read_table("ACCOUNT_RISK_PROFILES") cache.cache()`

Handling unknown accounts: New accounts (not in risk profiles) will have NULL profile fields after the left join. Treat them as moderate risk -- apply a base risk score of 15 for unknown accounts.

Level 3 Hints (Nearly Complete Guidance)

Complete foreachBatch flow: `python def process_batch(batch_df, batch_id, spark, sf): # 1. Parse JSON parsed = batch_df.select(from_json(col("value").cast("string"), schema).alias("t")).select("t.*") # 2. Get risk profiles (cached) profiles = get_cached_profiles(spark, sf) # 3. Broadcast join enriched = parsed.join(broadcast(profiles), ..., "left") # 4. Apply rules (withColumn for each rule) scored = apply_fraud_rules(enriched) # 5. Write live transactions sf.write(scored.select(...), "LIVE_TRANSACTIONS", "append") # 6. Write alerts where risk_score > 50 alerts = scored.filter(col("risk_score") > 50) sf.write(alerts.select(...), "FRAUD_ALERTS", "append")`

Alert details as JSON: Use `F.to_json(F.struct(...))` to build the VARIANT column: `python F.to_json(F.struct( col("profile_avg_amount"), col("profile_max_amount"), col("txn_count_5min"), col("rule_amount_threshold"), col("rule_location_anomaly"), )).alias("details")`

---

9. Validation Queries

Run these queries in Snowflake after completing the batch and streaming jobs.

Validate Batch Layer

```
-- 1. Check historical transactions loaded correctly
SELECT COUNT(*) AS total_txns,
       COUNT(DISTINCT account_id) AS unique_accounts,
       MIN(transaction_ts) AS earliest,
       MAX(transaction_ts) AS latest,
       SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraud_count,
       ROUND(AVG(amount), 2) AS avg_amount
FROM historical_transactions;
-- Expected: ~200 txns, ~50 accounts, Jan-Jun 2024, ~10 fraud, avg ~$200

-- 2. Check risk profiles
SELECT COUNT(*) AS profile_count,
       ROUND(AVG(avg_amount), 2) AS avg_of_averages,
       ROUND(AVG(total_txns), 1) AS avg_txns_per_acct,
       SUM(total_fraud_count) AS total_flagged_historical
FROM account_risk_profiles;
-- Expected: ~50 profiles

-- 3. Inspect a specific account profile
SELECT account_id, total_txns, avg_amount, max_amount, std_dev_amount,
       common_locations, common_categories, fraud_rate
FROM account_risk_profiles
WHERE account_id = 'ACC_001';
-- ACC_001 should show Chicago, IL as primary location
```

Validate Speed Layer

```
-- 4. Check live transactions are flowing
SELECT COUNT(*) AS live_count,
       MIN(transaction_ts) AS earliest_live,
       MAX(transaction_ts) AS latest_live,
       ROUND(AVG(risk_score), 1) AS avg_risk_score
FROM live_transactions;

-- 5. Check fraud alerts
SELECT COUNT(*) AS alert_count,
       ROUND(AVG(risk_score), 1) AS avg_risk_score,
       MAX(risk_score) AS max_risk_score
FROM fraud_alerts;

-- 6. See the most recent alerts with details
SELECT alert_id, transaction_id, account_id, amount,
       rule_triggered, risk_score, details, detected_at
FROM fraud_alerts
ORDER BY detected_at DESC
LIMIT 10;

-- 7. Rule breakdown
SELECT
   SUM(CASE WHEN rule_triggered ILIKE '%amount_threshold%' THEN 1 ELSE 0 END) AS amount_rules,
   SUM(CASE WHEN rule_triggered ILIKE '%location_anomaly%' THEN 1 ELSE 0 END) AS location_rules,
   SUM(CASE WHEN rule_triggered ILIKE '%velocity_check%' THEN 1 ELSE 0 END) AS velocity_rules,
   SUM(CASE WHEN rule_triggered ILIKE '%high_amount_low_avg%' THEN 1 ELSE 0 END) AS high_amt_rules
FROM fraud_alerts;
```

Validate Serving Layer

```
-- 8. Account 360 view
SELECT * FROM vw_account_360
WHERE total_fraud_alerts > 0
ORDER BY total_fraud_alerts DESC
LIMIT 10;

-- 9. Fraud summary by hour
SELECT * FROM vw_fraud_summary
ORDER BY alert_hour DESC
LIMIT 24;

-- 10. High risk accounts for investigation
SELECT account_id, risk_level, live_fraud_alerts, max_live_risk_score,
       historical_fraud_rate, top_rule
FROM vw_high_risk_accounts
ORDER BY risk_level, max_live_risk_score DESC;
```

Appendix: Key Concepts Reference

Lambda Architecture

Layer	Technology	Latency	Accuracy	Purpose
Batch	Glue + PySpark	Hours	High	Complete, accurate profiles
Speed	Spark Streaming	Seconds	Moderate	Real-time anomaly detection
Serving	Snowflake Views	Real-time	Combined	Unified query interface

Fraud Detection Rule Summary

Rule	Trigger Condition	Score	Rationale
amount_threshold	amount > 3x account average	25	Unusual spending magnitude
location_anomaly	City not in known locations	30	Geographic deviation
velocity_check	>5 transactions in 5 minutes	25	Rapid-fire pattern
high_amount_low_avg	amount > $2000 AND avg < $100	20	Spending profile mismatch

Risk Score Interpretation

Score Range	Classification	Action
0-25	LOW	No action; normal transaction
26-50	MEDIUM	Log for periodic review
51-75	HIGH	Send alert to fraud analyst

76-100	CRITICAL	Auto-block and notify customer